Module 10: Developing with Messaging Services

AWS Academy Cloud Developing

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 1: Introduction

Module 10: Developing with Messaging Services

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Module objectives

At the end of this module, you should be able to do the following:

- Illustrate how messaging services, including queues, pub/sub messaging, and streams, support asynchronous processing
- Describe Amazon Simple Queue Service (Amazon SQS)
- Send messages to an SQS queue
- Describe Amazon Simple Notification Service (Amazon SNS)
- Subscribe an SQS queue to an SNS topic
- Describe how Amazon Kinesis can be used for real-time analytics

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Module overview

Sections

1. Introduction
2. Processing requests asynchronously
3. Introducing Amazon SQS
4. Working with Amazon SQS messages
5. Configuring Amazon SQS queues
6. Introducing Amazon SNS
7. Developing with Amazon SNS
8. Introducing Kinesis Data Streams

Demonstration

- Working with Amazon Messaging Services

Lab

- Implementing a Messaging System

Knowledge check

aws

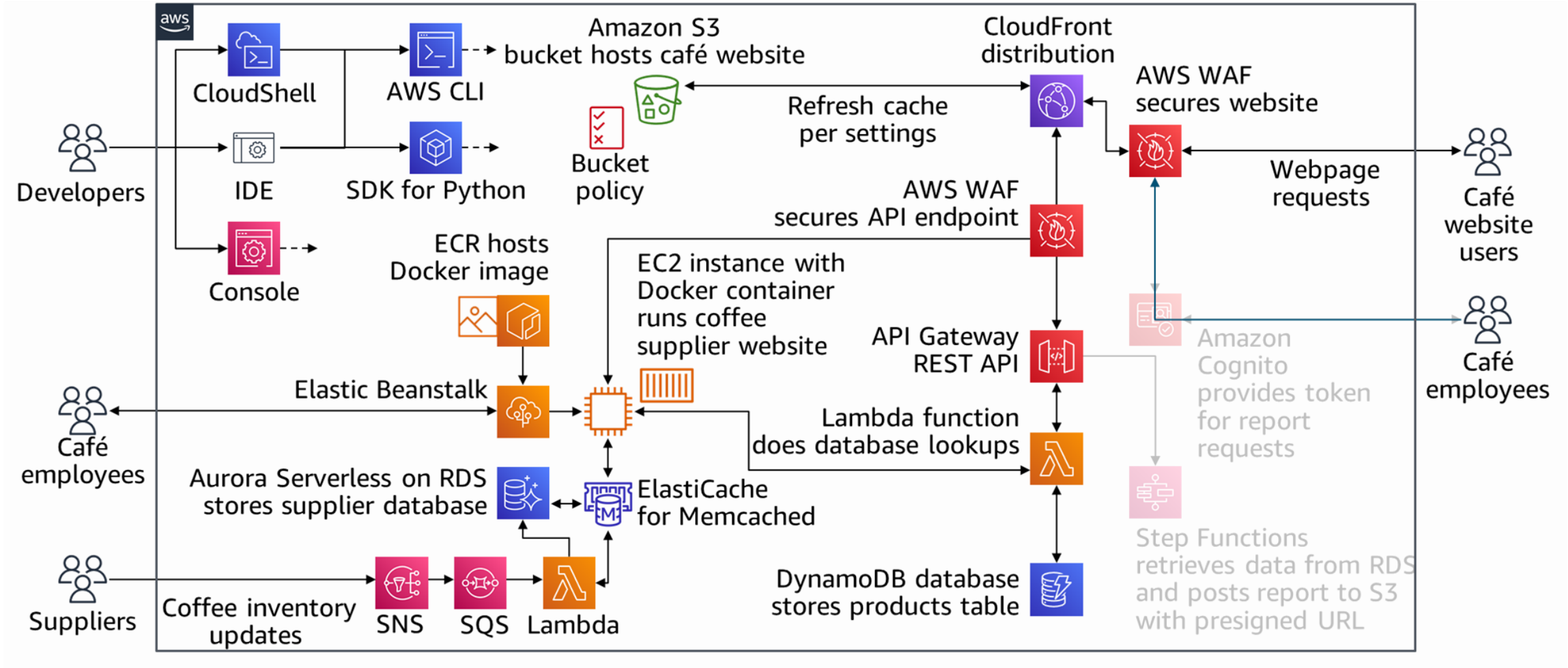
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Café business requirement

Currently, updating the coffee inventory on the website is a manual process. Sofía would like to find an easier way to keep the inventory updated. She considers setting up a messaging system to automatically receive and process inventory updates.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Messaging services as part of developing a cloud application



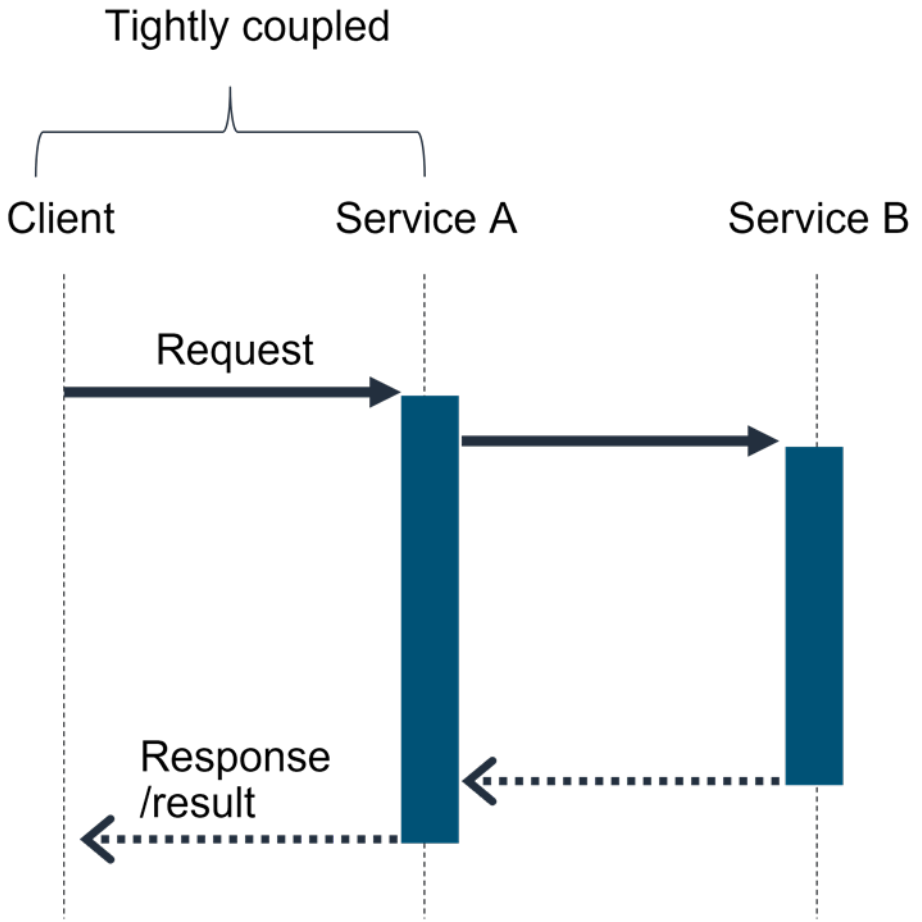
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 2: Processing requests asynchronously

Module 10: Developing with Messaging Services

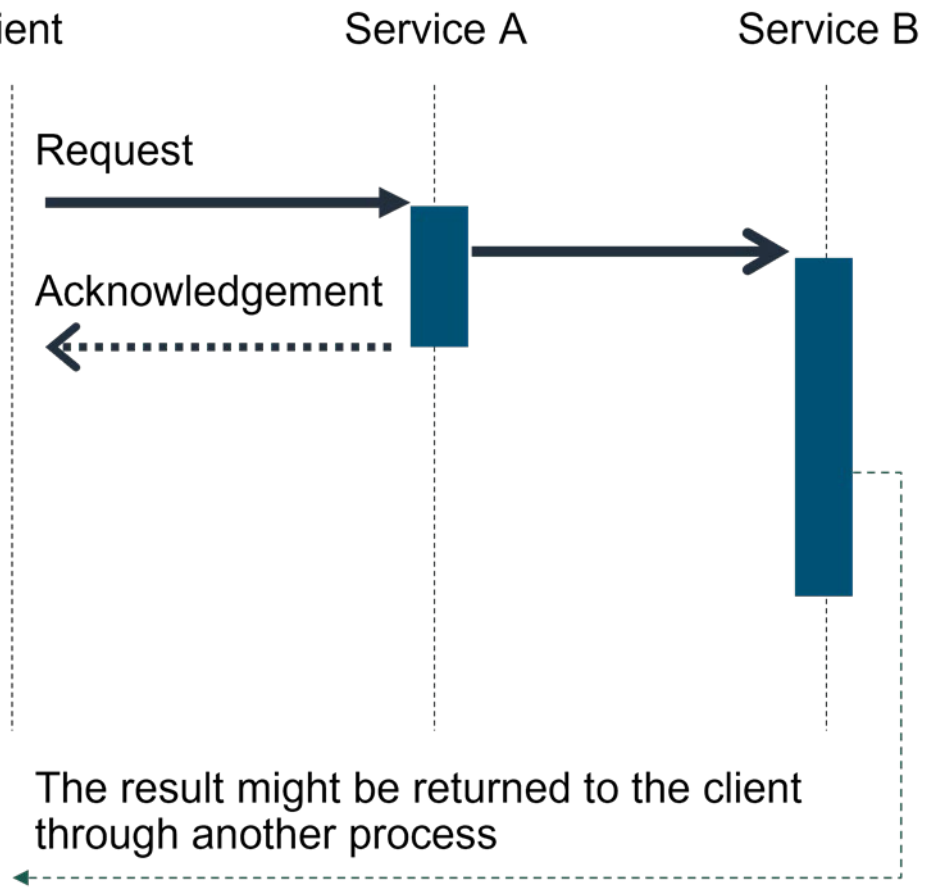
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Synchronous processing



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Asynchronous processing



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Asynchronous processing with messaging services

Message queues

Pub/sub messaging

Data streams

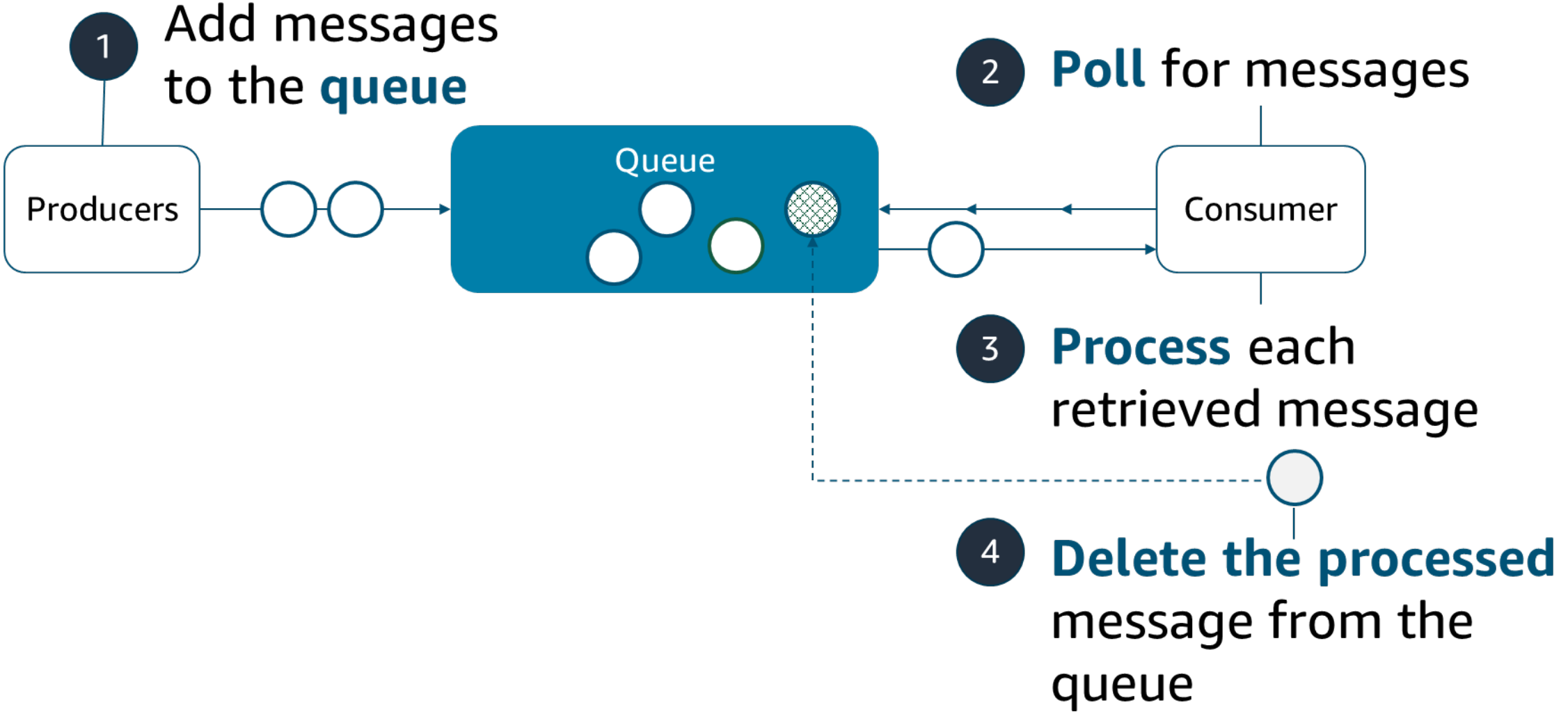
Amazon SQS

Amazon SNS

Amazon Kinesis Data Streams

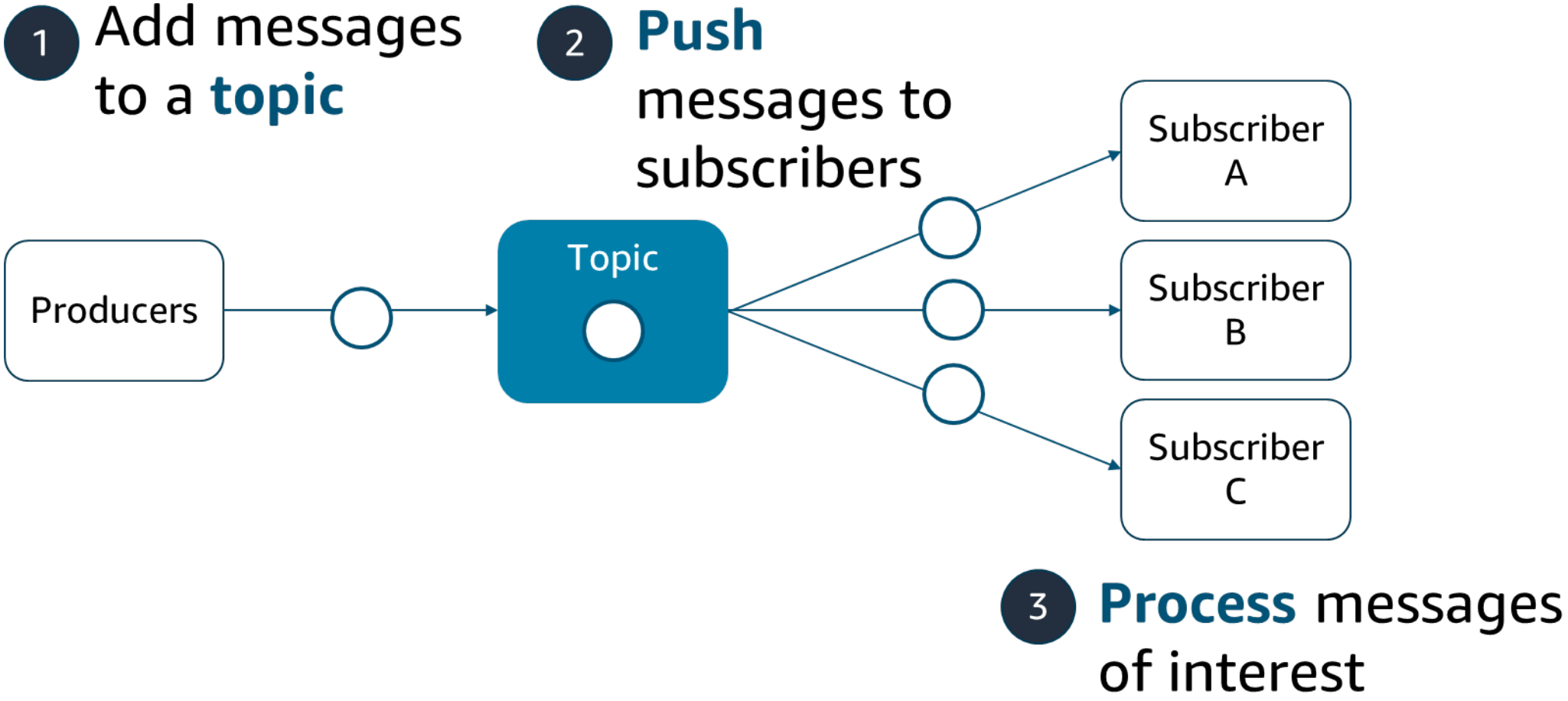
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Message queues overview



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Pub/sub messaging overview

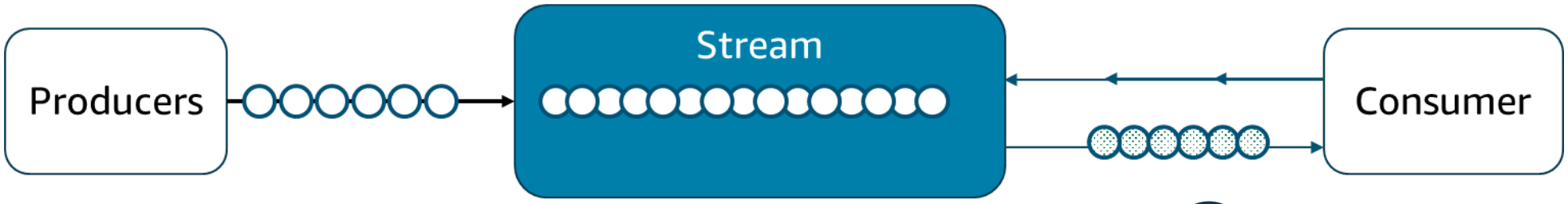


©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Data streams overview

1 Add messages to the **stream**

2 **Poll** for messages



3 **Aggregate** and analyze large numbers of messages

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 2 key takeaways

- An asynchronous design **reduces interdependencies** and can **improve responsiveness** to the client.
- **Messaging services** are good for creating asynchronous designs and include **queues, pub/sub messaging**, and **streams**.
- AWS provides **fully managed** messaging services including **Amazon SQS**, **Amazon SNS**, and **Kinesis Data Streams**.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 3: Introducing Amazon SQS

Module 10: Developing with Messaging Services

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Amazon SQS

Amazon Simple Queue Service (Amazon SQS)

Fully managed message queuing service that eliminates the complexity and overhead associated with managing and operating message queues

- Limits administrative overhead
- Reliably delivers messages at very high volume and throughput
- Scales dynamically
- Can keep data secure
- Offers standard and First-In-First-Out (FIFO) options

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Standard and FIFO queue types

Standard (default)

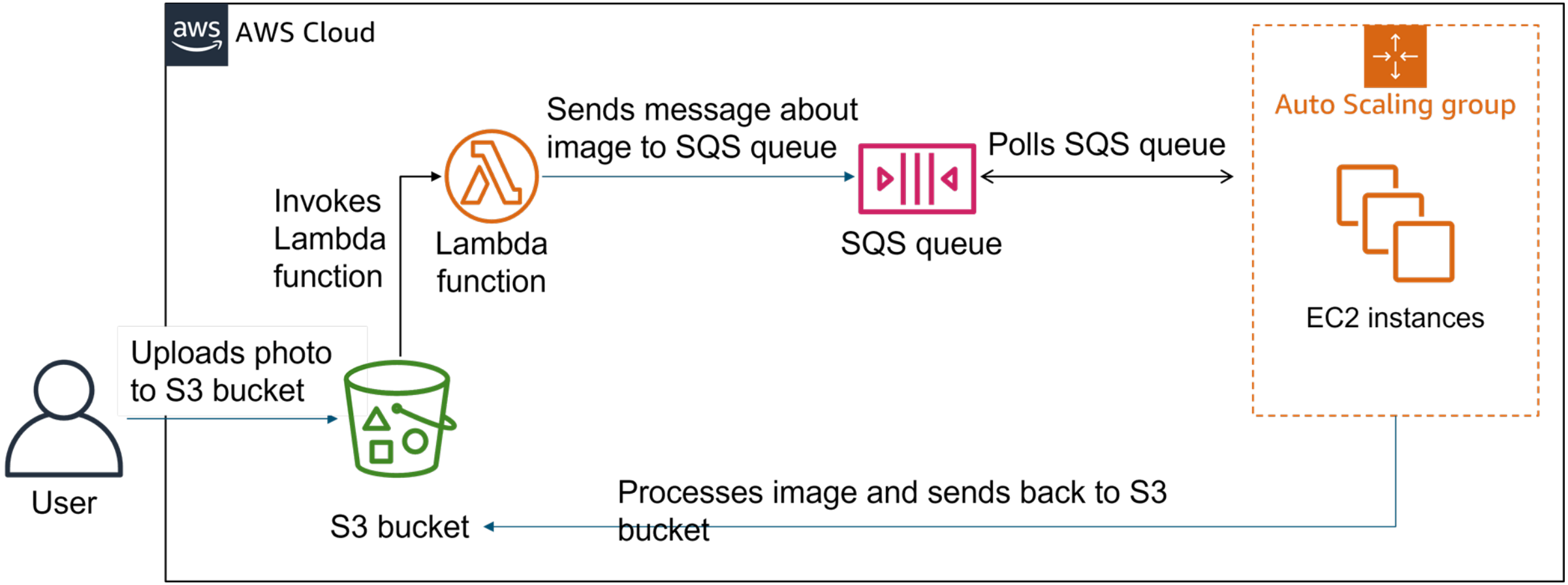
- Best effort ordering, at-least-once delivery. Nearly unlimited throughput.
- Use Cases
  - Let users upload media while resizing or encoding it
  - Process a high number of credit card validation requests
  - Schedule multiple entries to be added to a database that are not order dependent

FIFO

- Message order is preserved, exactly-once delivery. More limited throughput.
- Use Cases
  - Ensure that a deposit is recorded before a bank withdrawal happens
  - Ensure that a credit card transaction is not processed more than once
  - Prevent a student from enrolling in a course before they register for an account

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Amazon SQS architecture example



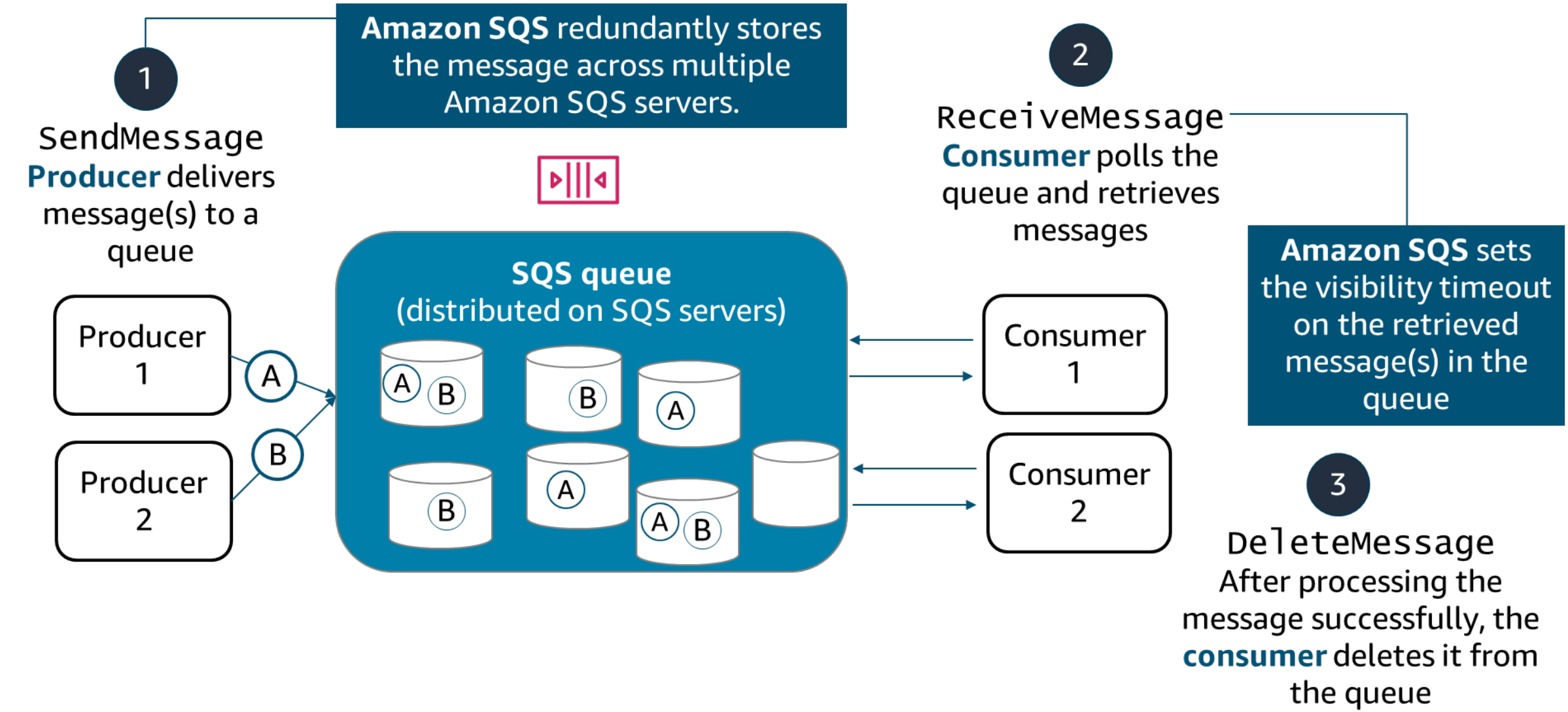
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 4: Working with Amazon SQS messages

Module 10: Developing with Messaging Services

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Amazon SQS message lifecycle



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

The SendMessage operation

Example AWS CLI command for **SendMessage** operation

```
aws sqs send-message
--queue-url https://sqs.us-east-1.amazonaws.com/80398EXAMPLE/MyQueue
--message-body "My first message"
```

```
{
"MD5OfMessageBody": "51b0a325...39163aa0",
"MessageId": "d6790f8d-d575-4f01-bc51-40122EXAMPLE"
}
```

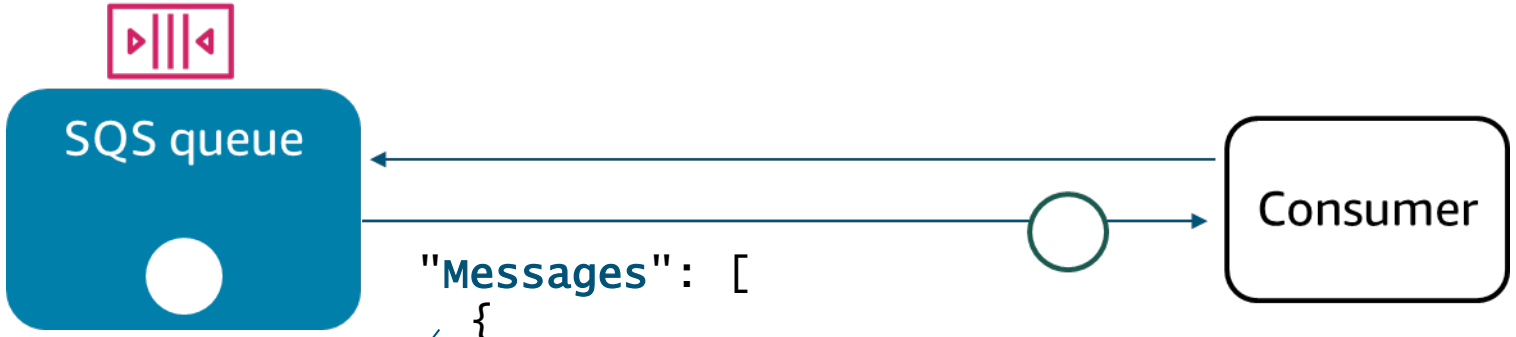
Example Amazon SQS **response** with **MD5 hash** of body and **MessageID**

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

The ReceiveMessage operation

```
aws sqs receive-message
--queue-url https://sqs.us-east1.amazonaws.com/80398EXAMPLE/MyQueue
--max-number-of-messages 10
```

Example AWS CLI command for **ReceiveMessage** operation

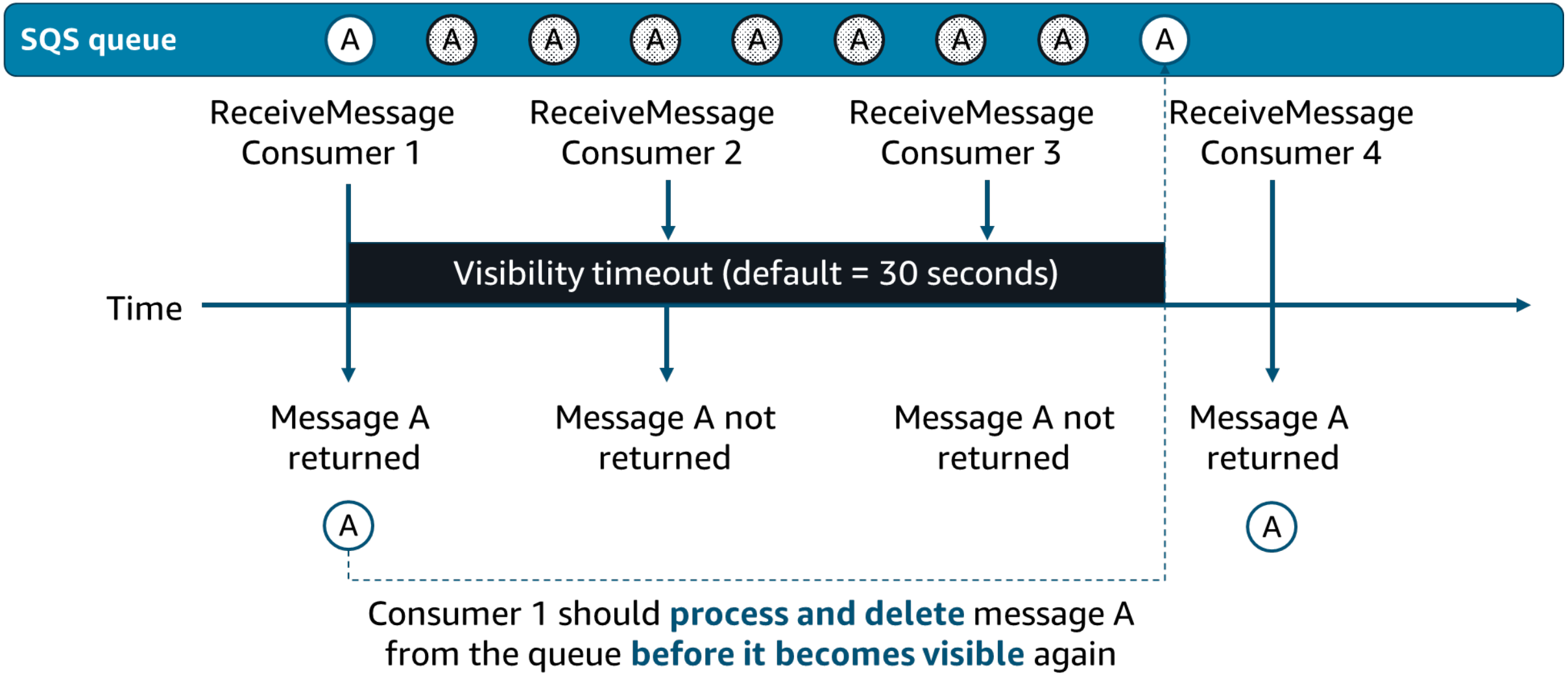



```
"Messages": [
  {
    "Body": "My first message.",
    "ReceiptHandle": "AQEBzbVv...fqNzFw==",
    "MD5OfBody": "51b0a325...39163aa0",
        "MessageId": "d6790f8d-d575-4f01-bc51-40122EXAMPLE",
    "Attributes": {
      …
    },
  }
]
}
```

Example Amazon SQS response with **messages**. Each message has a **ReceiptHandle**.

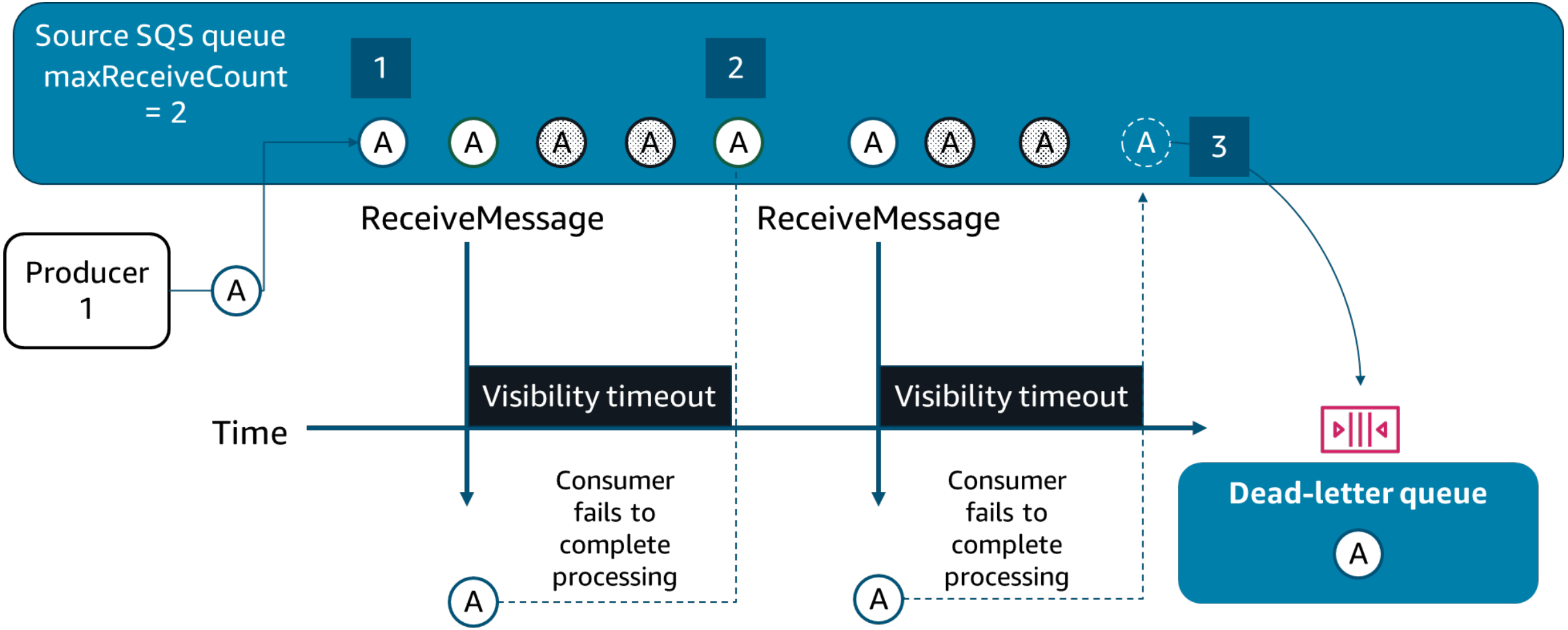
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

ReceiveMessage and visibility timeout



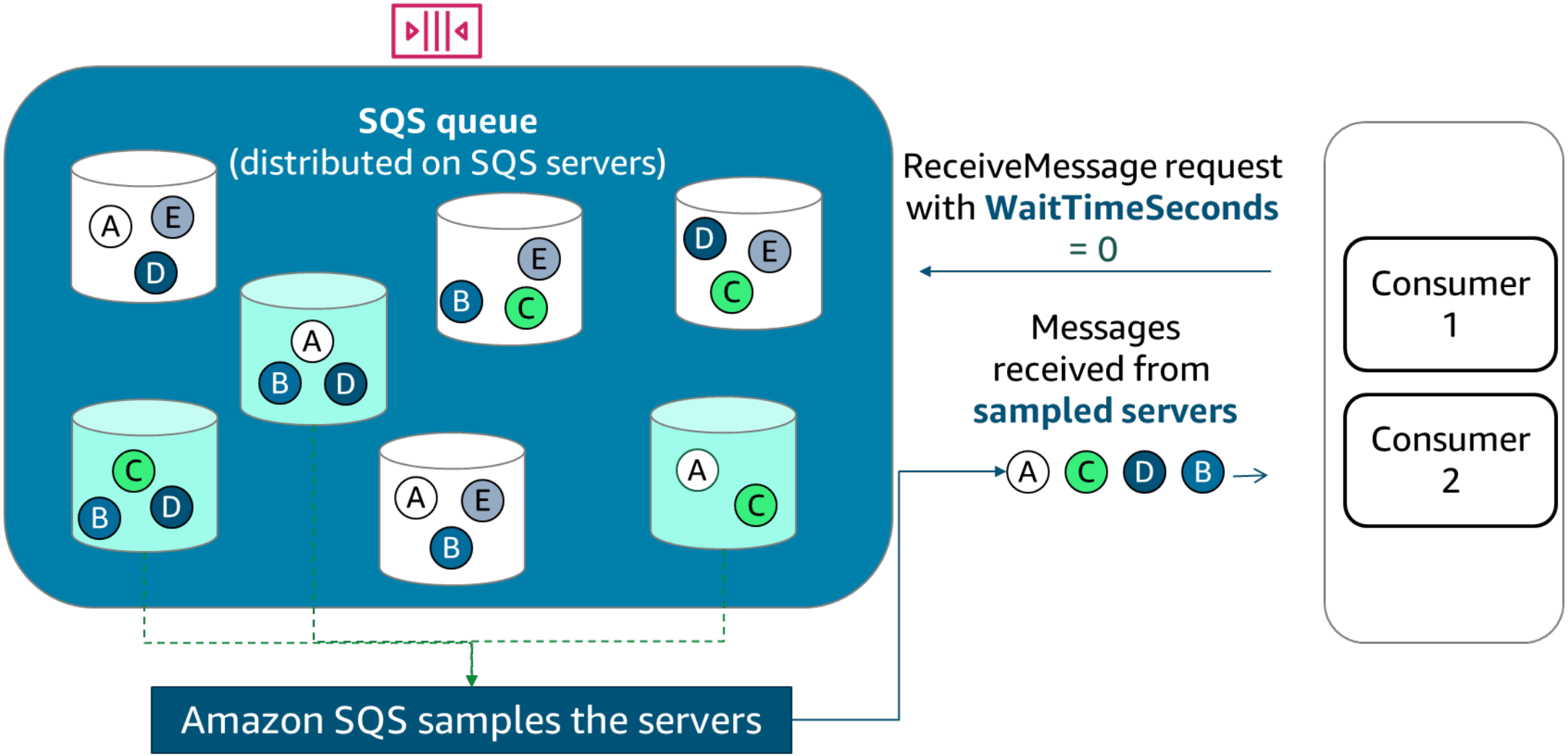
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Configuring a dead-letter queue



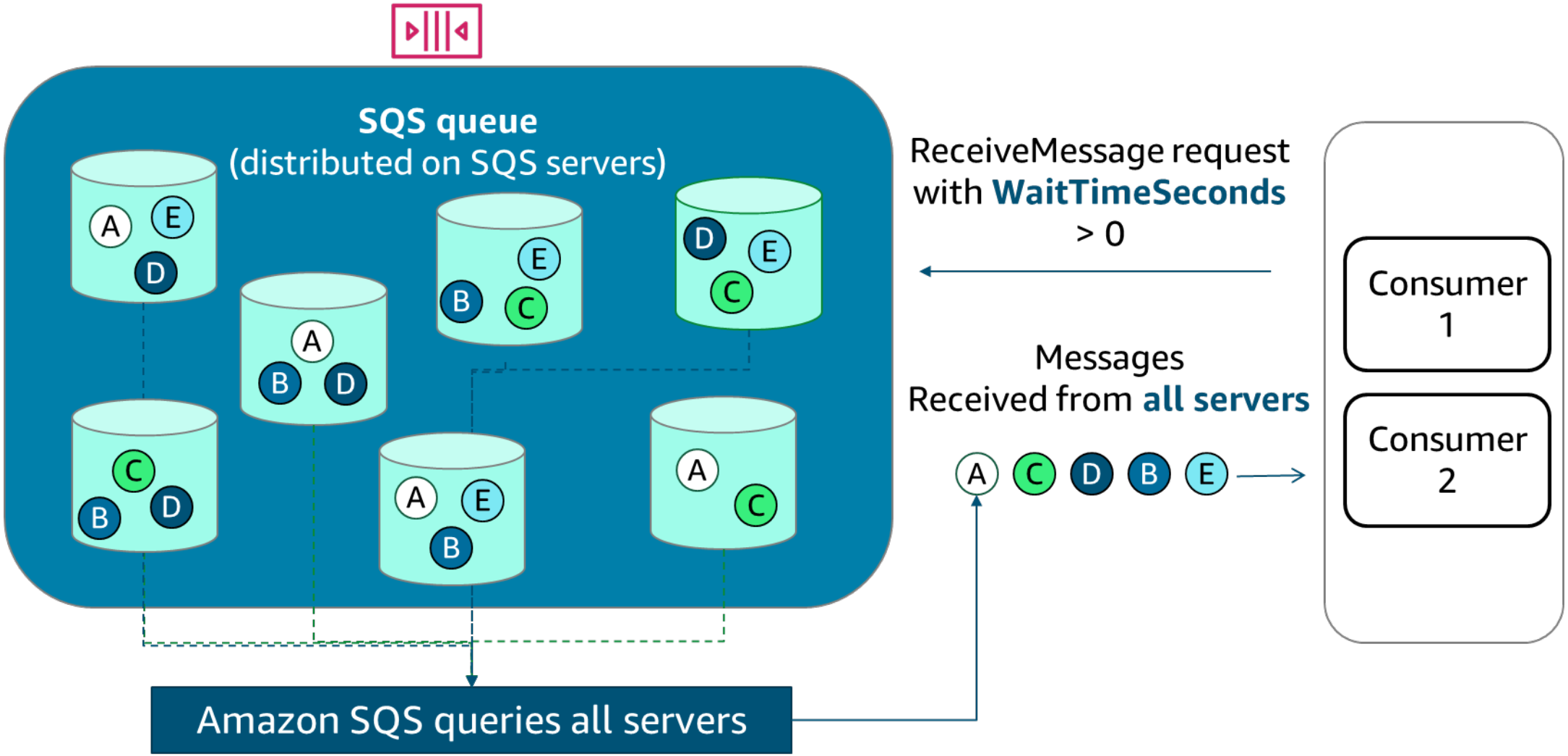
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

ReceiveMessage and short polling



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

ReceiveMessage and long polling



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

ReceiveMessage long polling example

Retrieve messages from "testQueue" at this URL

```
https://sqs.us-east-1.amazonaws.com/123456789012/testQueue/
?Action=ReceiveMessage
&WaitTimeSeconds=10
&MaxNumberOfMessages=5
&VisibilityTimeout=15
&AttributeName=All;
&Expires=2019-04-18T22%3A52%3A43PST
&Version=2012-11-05
&AUTPARAMS
```

Use long polling with a wait time of 10 seconds

Retrieve up to 5 messages at a time

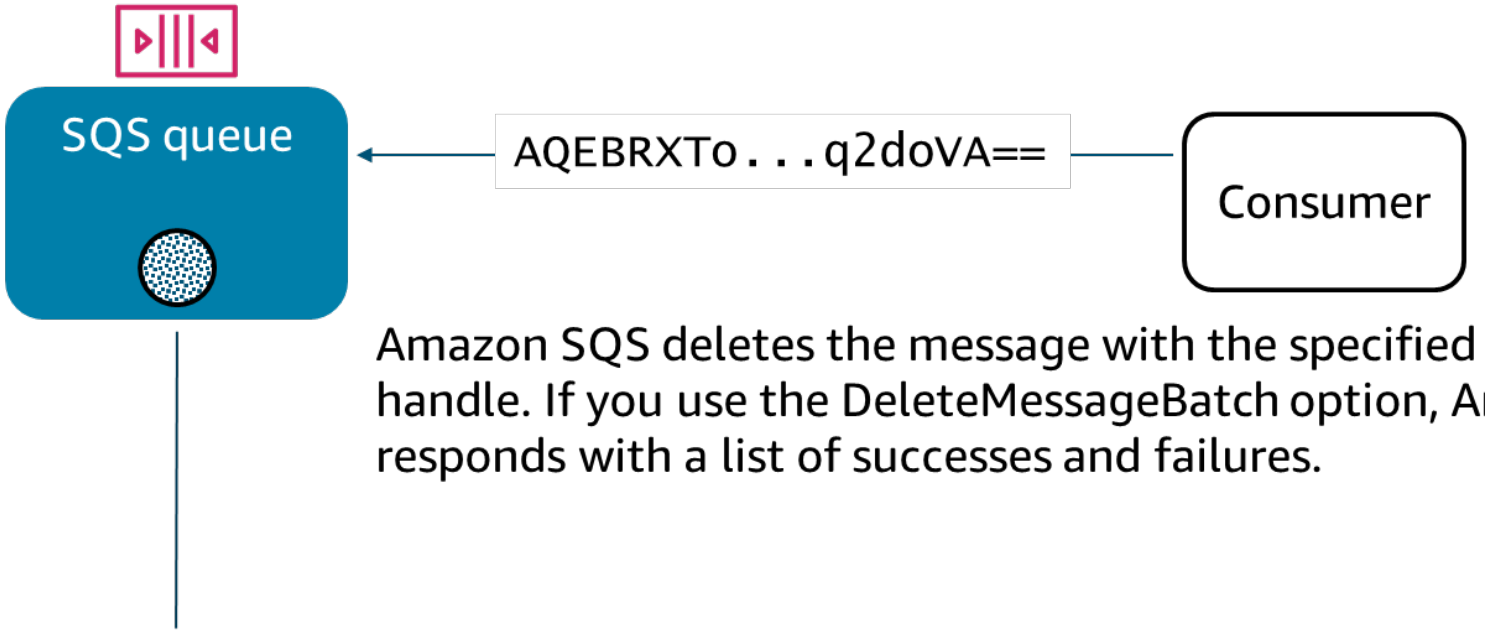
Make these messages invisible to other consumers for 15 seconds

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

DeleteMessage operation

```
aws sqs delete-message
--queue-url https://sqs.us-east-1.amazonaws.com/80398EXAMPLE/MyQueue
--receipt-handle AQEBRXTo...q2doVA==
```

Example AWS CLI command for **DeleteMessage** operation



Amazon SQS deletes the message with the specified receipt handle. If you use the DeleteMessageBatch option, Amazon SQS responds with a list of successes and failures.

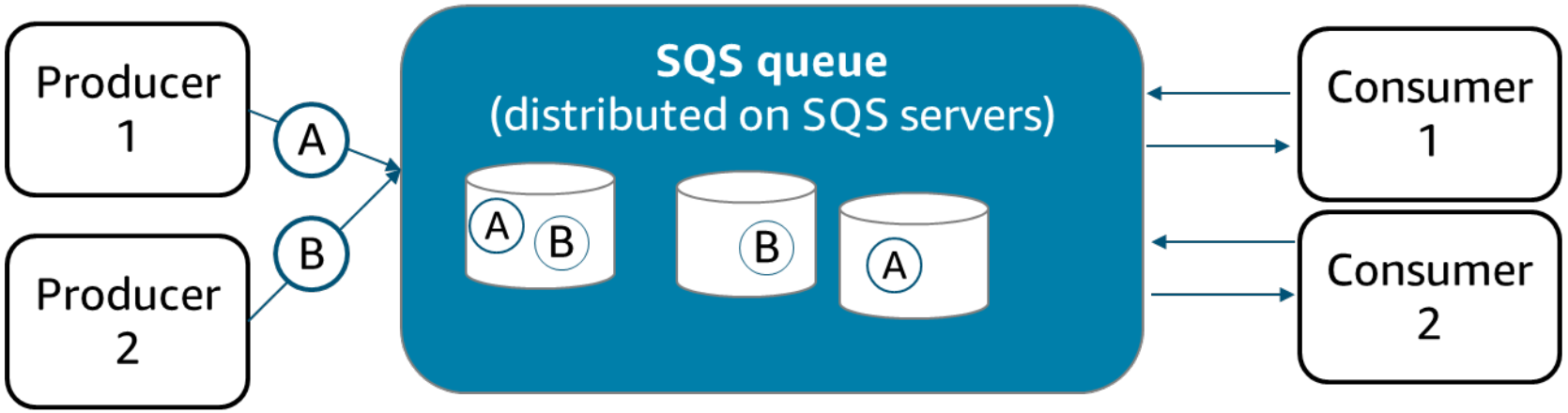
Amazon SQS automatically deletes messages that remain in queue beyond the queue's **retention period**.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Amazon SQS message lifecycle operations summary

1

```
aws sqs send-message
--queue-url https://sqs.us-east-
1.amazonaws.com/80398EXAMPLE/MyQueue
--message-body "My first message"
```



2

```
aws sqs receive-message
--queue-url
https://sqs.us-
east1.amazonaws.com/80398
EXAMPLE/MyQueue
--max-number-of-messages
1
```

3

```
aws sqs delete-message
--queue-url https://sqs.us-east-
1.amazonaws.com/80398EXAMPLE/MyQueue
--receipt-handle AQEBRXTo...q2doVA==
```

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 4 key takeaways

- Developers use the **SendMessage, ReceiveMessage**, and **DeleteMessage** API operations to add, retrieve, and delete queue messages.
- Amazon SQS makes a retrieved message invisible on the queue for the duration of the **visibility timeout**.
- Amazon SQS can send failed records to a **dead-letter queue** for separate processing.
- **Short polling** samples Amazon SQS servers for messages, while **long polling** queries all servers.

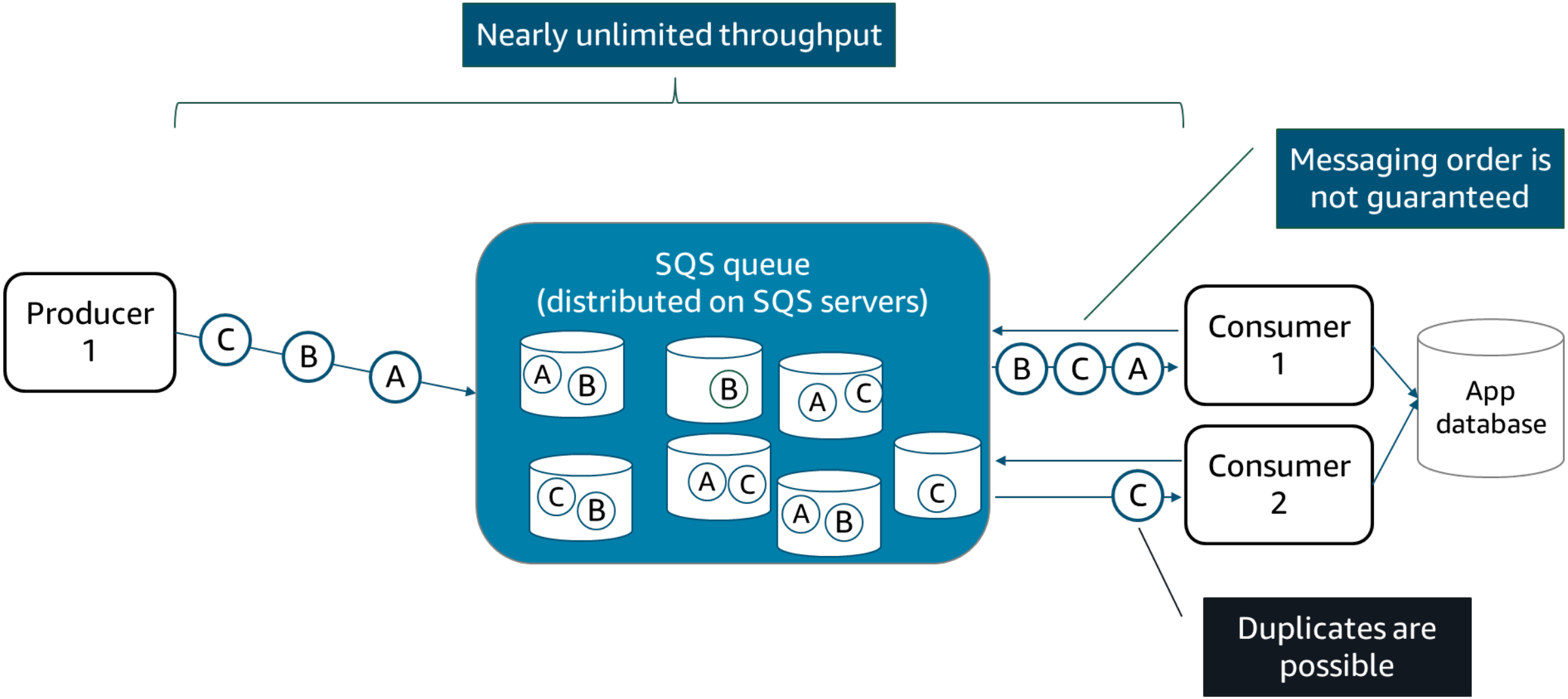
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 5: Configuring Amazon SQS queues

Module 10: Developing with Messaging Services

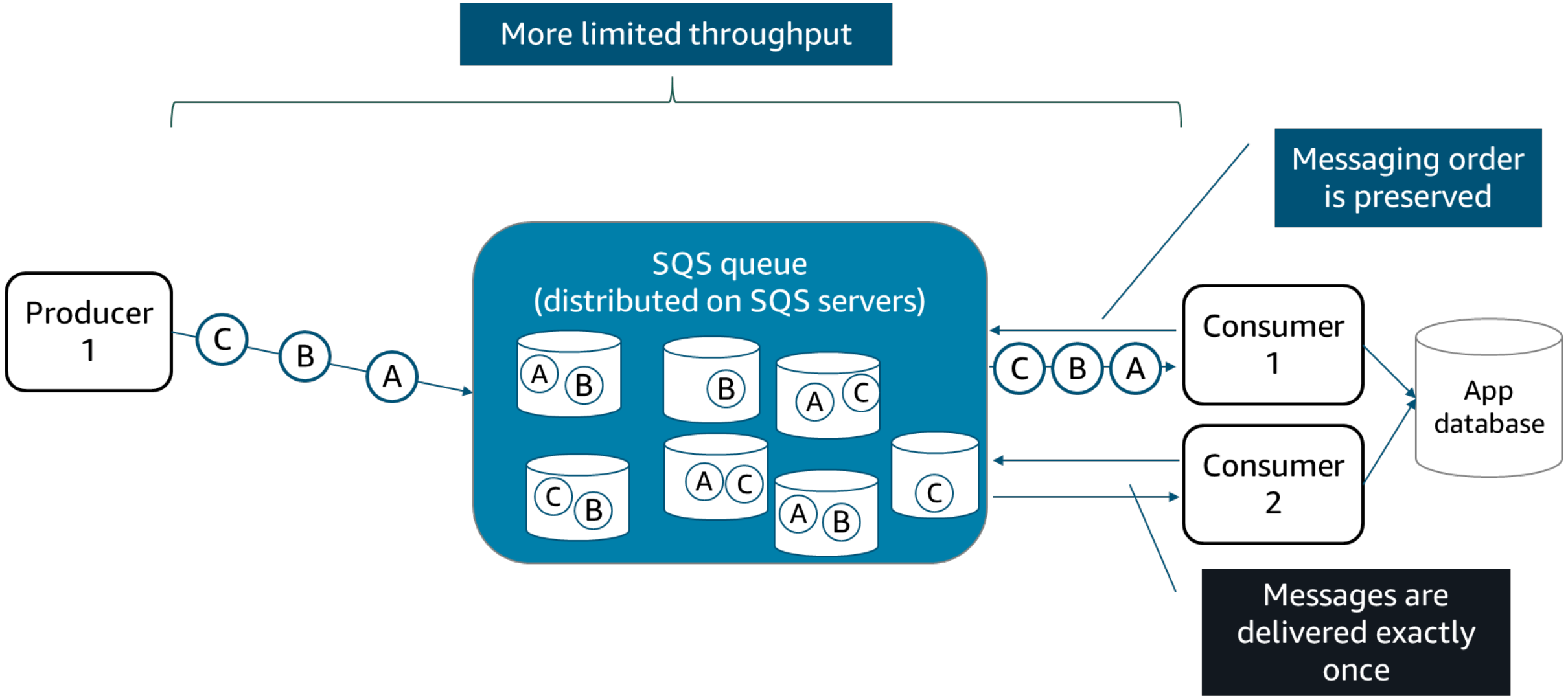
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Standard SQS queues



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

FIFO SQS queues



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Creating and configuring queues

API operation	Description
CreateQueue	Creates an SQS queue including queue attributes. If you don't provide a value for an attribute, the queue is created with the default value for the attribute.
GetQueueAttributes	Retrieves attributes of a queue including those that help you estimate the resources required to process queue messages; for example, the approximate number of messages available to retrieve and how many are currently invisible.
SetQueueAttributes	Sets attributes on an existing queue.
GetQueueUrl	Retrieves the URL of an SQS queue.
ListQueues	Retrieves a list of your queues.
DeleteQueue	Deletes a queue, regardless of whether the queue is empty. When a queue is deleted, any messages in the queue are no longer available.

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Create SQS queue example

Create a queue using the attributes in create-queue.json

```
aws sqs create-queue --queue-name MyQueue --attributes file://create-queue.json
```

create-queue.json

```
{
  "RedrivePolicy": "{\"deadLetterTargetArn\":\"arn:aws:sqs:us-east-
1:80398EXAMPLE:MyDeadLetterQueue\",\"maxReceiveCount\":\"1000\"}",
  "MessageRetentionPeriod": "259200"
}
```

Set the dead-letter queue destination

Set the message retention period to 3 days

Set the maximum receive count to 1,000

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Three types of SQS queue security

Identity and access management

AWS Identity and Access Management (**IAM**) and Amazon **SQS policies**

Data encryption

Server-side encryption (**SSE**) with AWS Key Management Service (**AWS KMS**)

Internetwork traffic privacy

Amazon Virtual Private Cloud (**Amazon VPC**) **endpoint**

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Examples of IAM and Amazon SQS policies

Equivalent permissions to MyQueue

IAM policy attached to IAM user Bob	Amazon SQS policy attached to MyQueue
Allow	Allow who:
	Bob
Actions:	Actions:
ReceiveMessage, SendMessage	ReceiveMessage, SendMessage
Resource:	Resource:
arn:aws:sqs:us-east-1:80398EXAMPLE:MyQueue	arn:aws:sqs:us-east-1:80398EXAMPLE:MyQueue

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 5 key takeaways

- **Standard** queues provide nearly **unlimited throughput** but don't preserve message order and might include duplicate messages.
- **FIFO** queues **preserve order** and provide **exactly-once delivery** but have **more limited throughput**.
- Queue security includes **managing access** to queue resources, **protecting data** stored in the queue, and **limiting exposure** of messages to the internet.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 6: Introducing Amazon SNS

Module 10: Developing with Messaging Services

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Amazon SNS

Amazon Simple Notification Service (Amazon SNS)

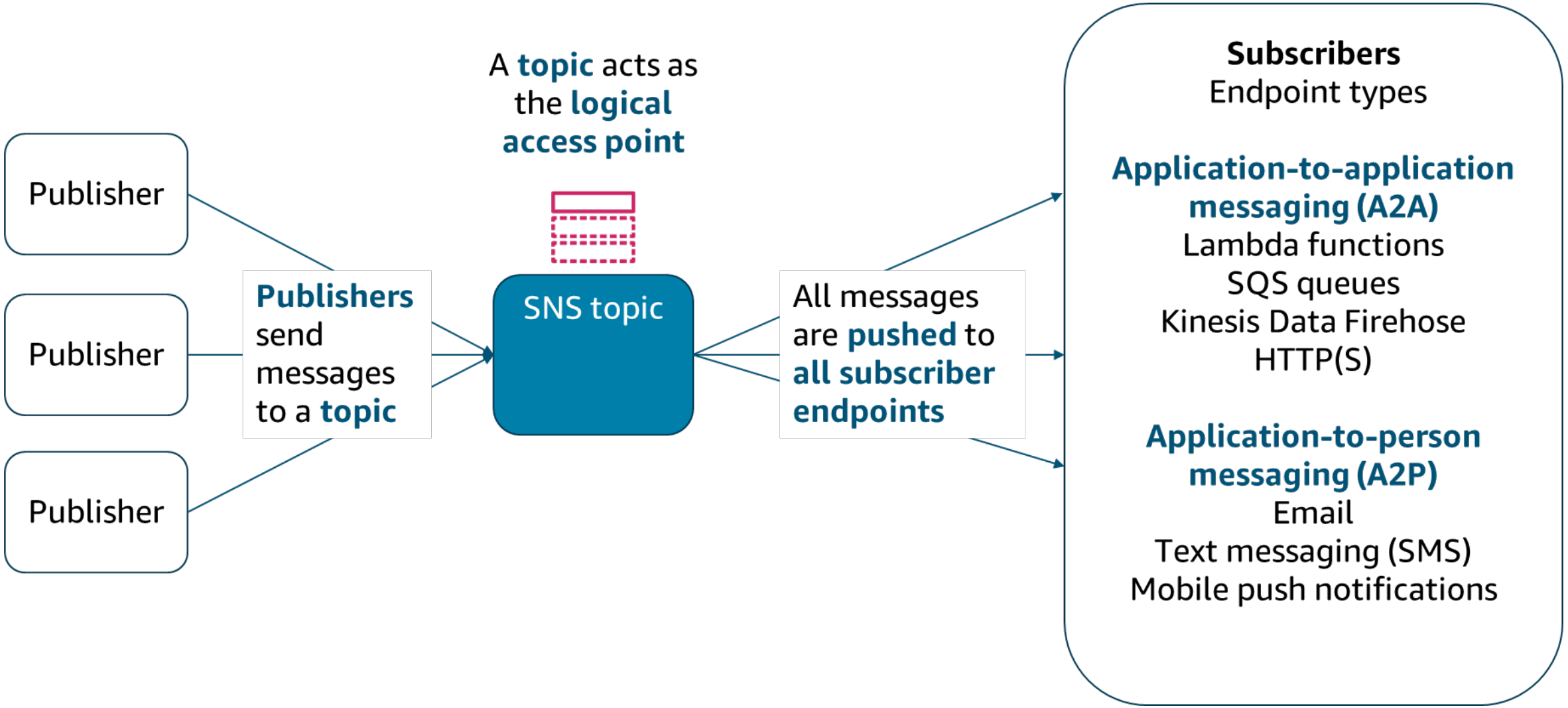
Fully managed messaging service with pub/sub functionality for many-to-many messaging between distributed systems, microservices, and event-driven applications

- Offloads message filtering logic from your subscriber systems and message routing logic from your publisher systems
- Reliably delivers messages and provides failure management features (retries, dead-letter queues)
- Scales dynamically
- Offers a FIFO topics option, which works with FIFO SQS queues to preserve message order

aws

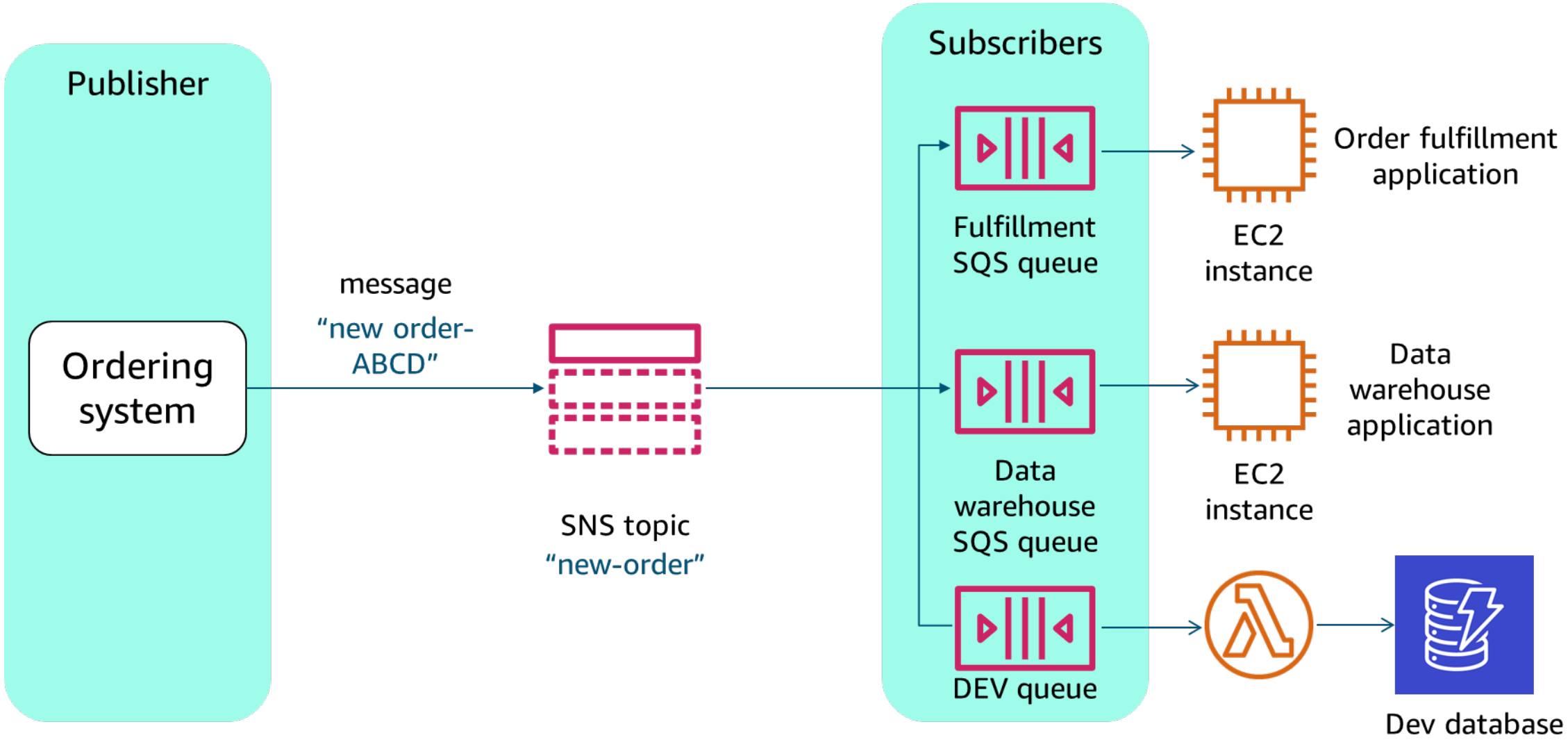
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Amazon SNS overview



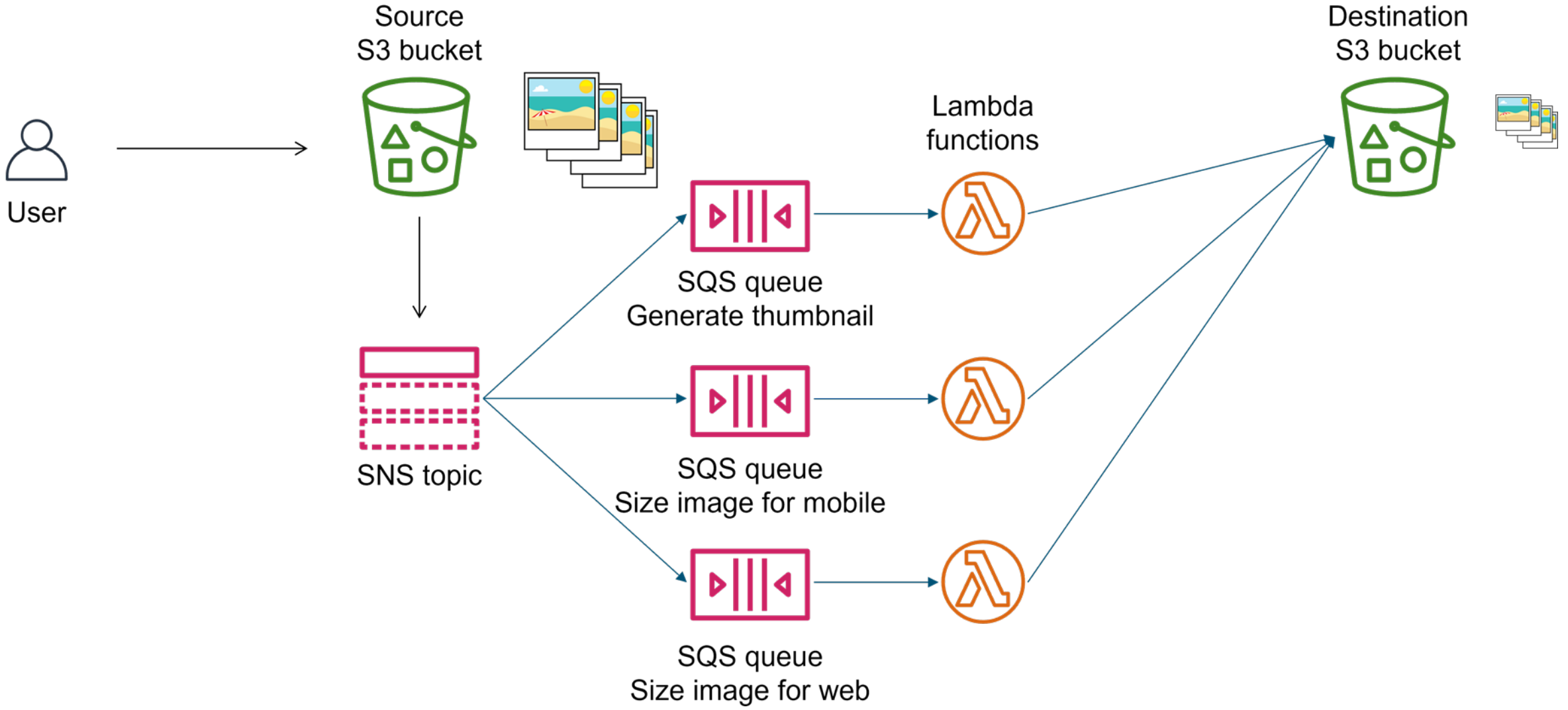
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Amazon SNS fanout pattern example



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Image processing fanout example



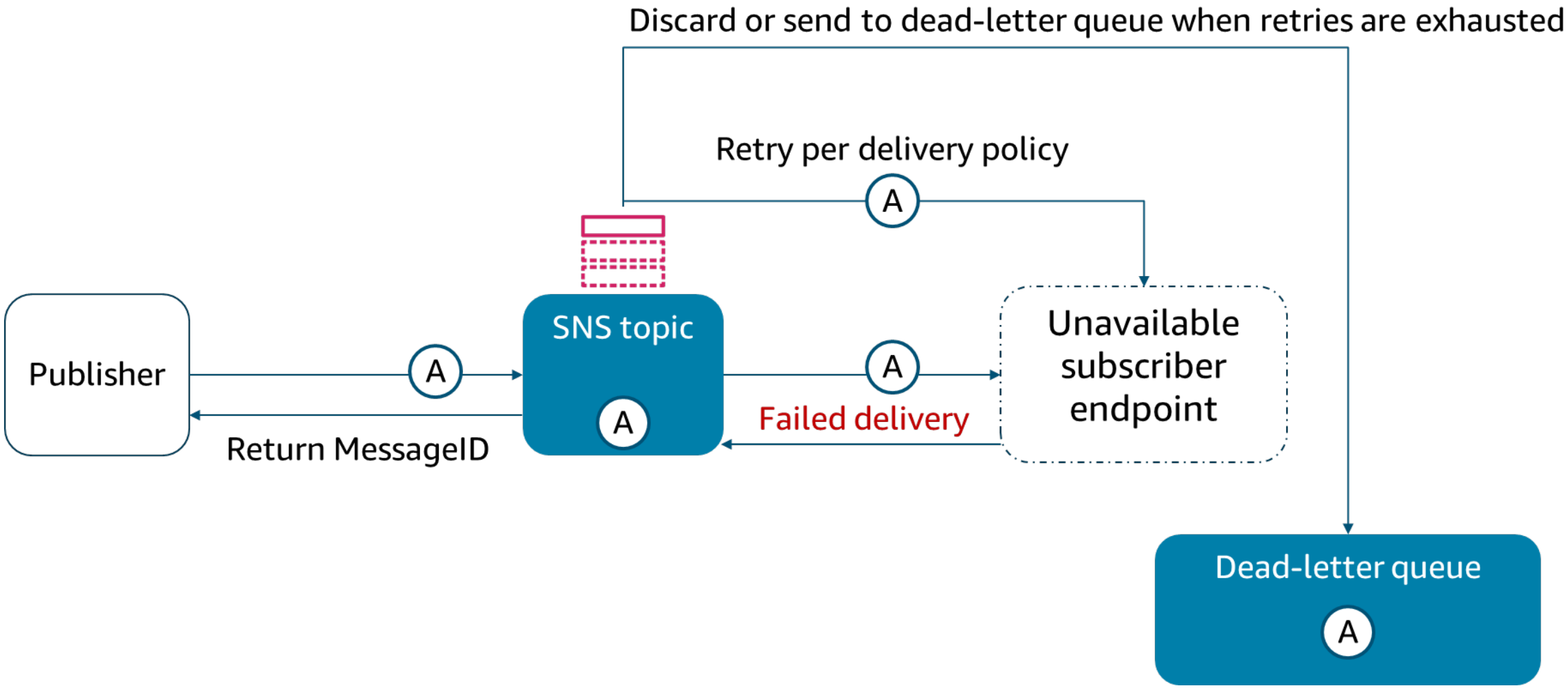
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 7: Developing with Amazon SNS

Module 10: Developing with Messaging Services

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Amazon SNS message delivery lifecycle



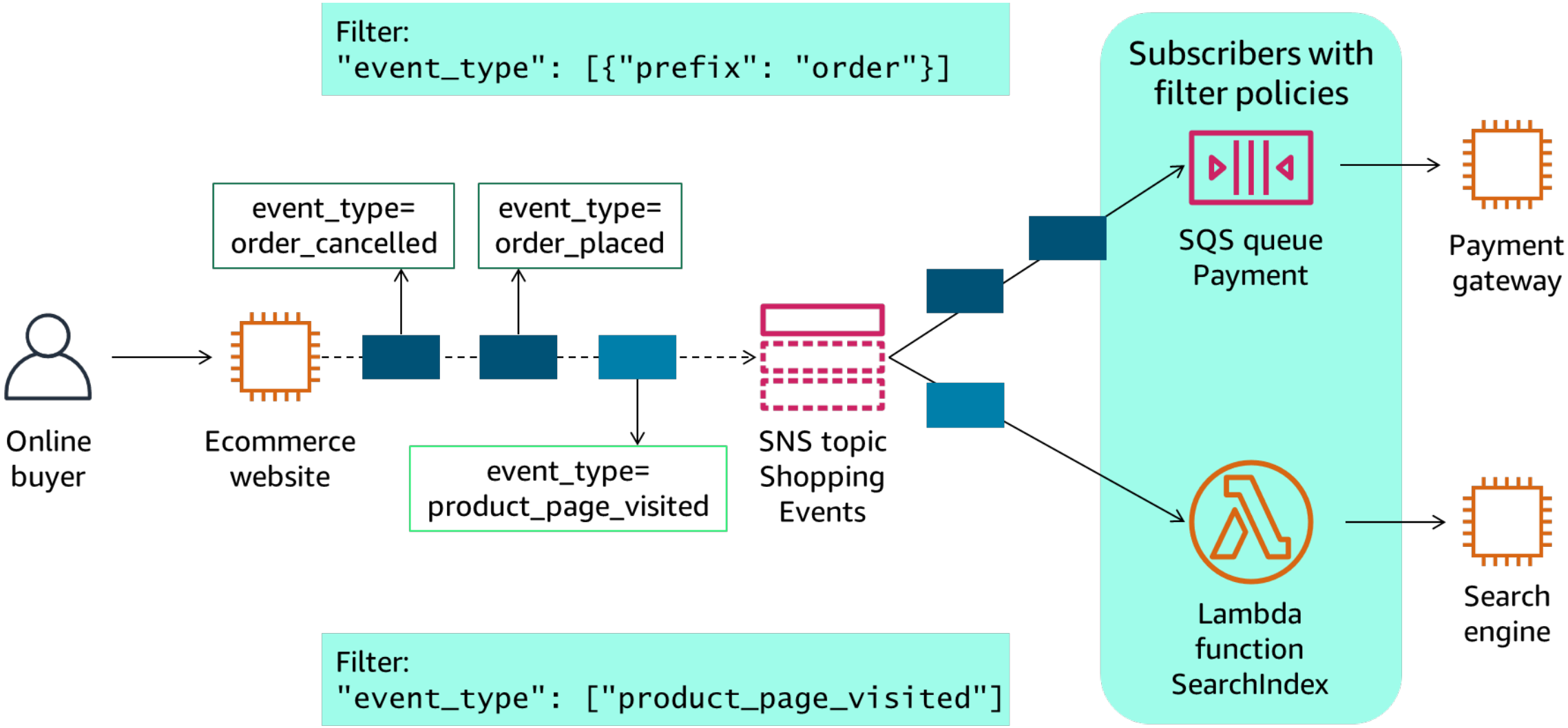
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Amazon SNS API operations

API operation	Description	Input	Output
`CreateTopic`	Creates a topic	• Topic name	• ARN of topic
`Subscribe`	Prepares to subscribe to an endpoint	• Subscriber’s endpoint • Protocol • ARN of topic	• ARN of topic
`ConfirmSubscription`	Verifies an endpoint owner's intent to receive messages	• Token sent to endpoint	
`Publish`	Sends a message to all of a topic's subscribed endpoints	• Message • Message attributes (optional) • Message structure:json (optional) • Subject (optional) • ARN of topic	• Message ID
`DeleteTopic`	Deletes a topic and all of its subscriptions	• ARN of topic	

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Amazon SNS message filtering



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Subscription filter policy example

```
topic_arn = sns.create_topic(
        Name='ShoppingEvents'
)['TopicArn']


search_engine_subscription_arn = sns.subscribe(
        TopicArn = topic_arn,
        Protocol = 'lambda',
        Endpoint = 'arn:aws:lambda:us-east-1:123456789012:function:SearchIndex'
)['SubscriptionArn']

sns.set_subscription_attributes(
        SubscriptionArn = search_engine_subscription_arn,
        AttributeName = 'FilterPolicy'
        AttributeValue = '{"event_type": ["product_page_visited"]}'
)
```

SNS topic

Subscriber

Attribute in filter policy

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Message with attributes example

```
message = '{"product": {"id": 1251, "status": "in_stock"},'\
          ' "buyer": {"id":4454}}'

sns.publish(
        TopicArn = topic_arn,
        Subject = "Product Visited #1251",
        Message = message,
        MessageAttributes = {
                'event_type': {
                        'DataType': 'String',
                        'StringValue': 'product_page_visited'
                }
        }
}
```

Message

Message attribute that matches the attribute in the filter policy

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Three types of Amazon SNS security

Identity and access management

IAM and Amazon **SNS policies**

Data encryption

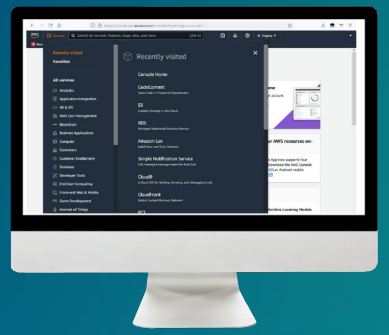
Server-side encryption **(SSE)** with **AWS KMS**

Internetwork traffic privacy

Amazon **VPC endpoint**

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Demonstration: Working with Amazon Messaging Services



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Sections 6 and 7 key takeaways

- Developers **publish** messages to SNS **topics**, and **subscribers** to that topic get copies of all published messages.
- With **attribute filtering**, subscribers receive **only relevant messages**.
- Subscriber **endpoints** include both application-to-application (**A2A**) and application-to-person (**A2P**) types.
- Amazon SNS security includes **managing access** to SNS resources, **protecting data** sent to topics, and **limiting exposure** of messages to the internet.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 8: Introducing Kinesis Data Streams

Module 10: Developing with Messaging Services

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Kinesis Data Streams

Amazon Kinesis Data Streams

Fully managed, massively scalable and durable real-time data streaming service that you can use to ingest, buffer, and process streaming data in real time

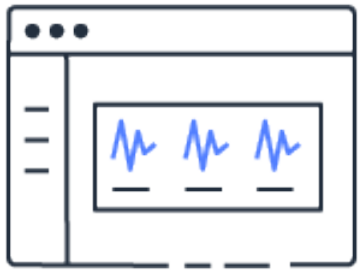
- Continuously capture gigabytes of data per second
- Source stream data from sources including website clickstreams, database event streams, financial transactions, social media feeds, IT logs, and location-tracking events
- Easily process data with built-in integrations with Lambda and other Kinesis services such as Kinesis Data Analytics and Kinesis Data Firehose

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Kinesis Data Streams use cases

Log and event data collection



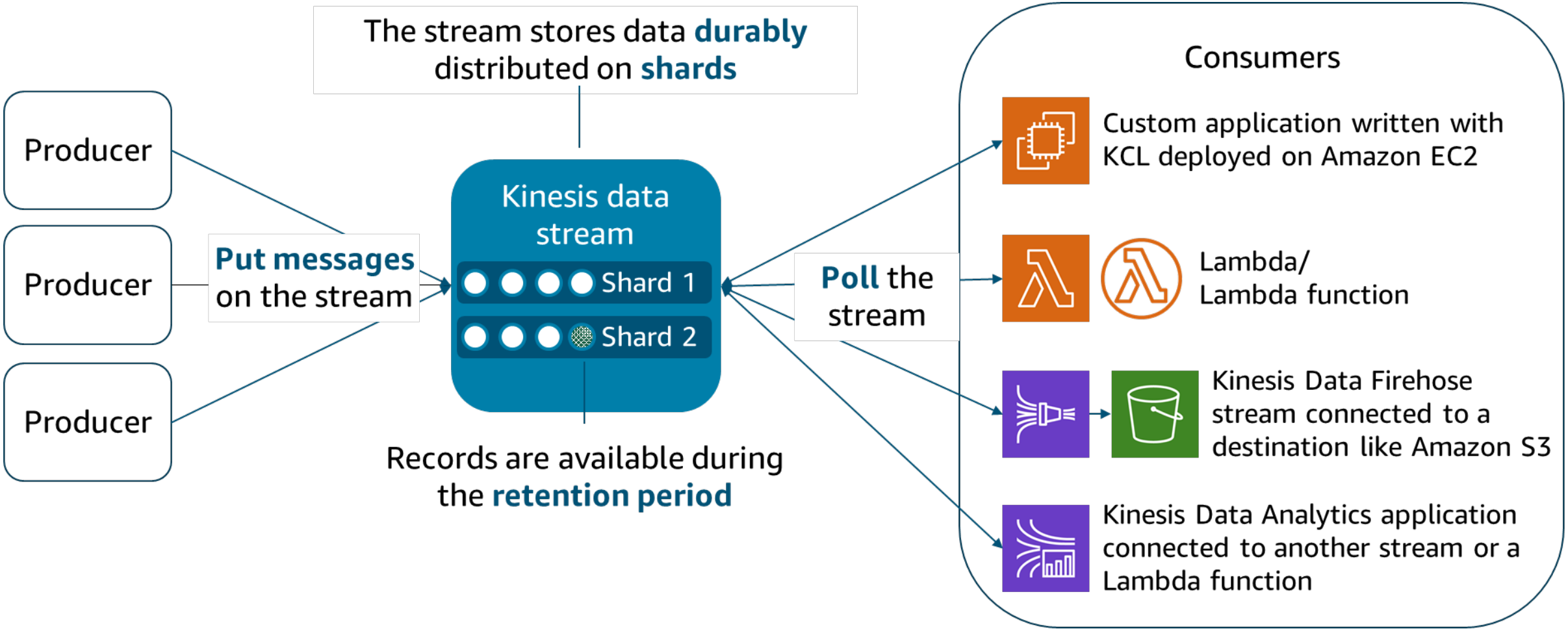
Real-time analytics

Mobile data capture

Gaming data feed

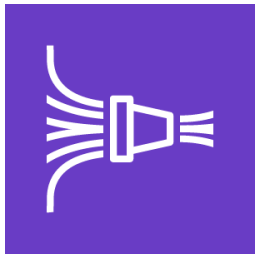
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Kinesis Data Streams overview



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Other Kinesis data streaming services



Amazon Kinesis Data Firehose

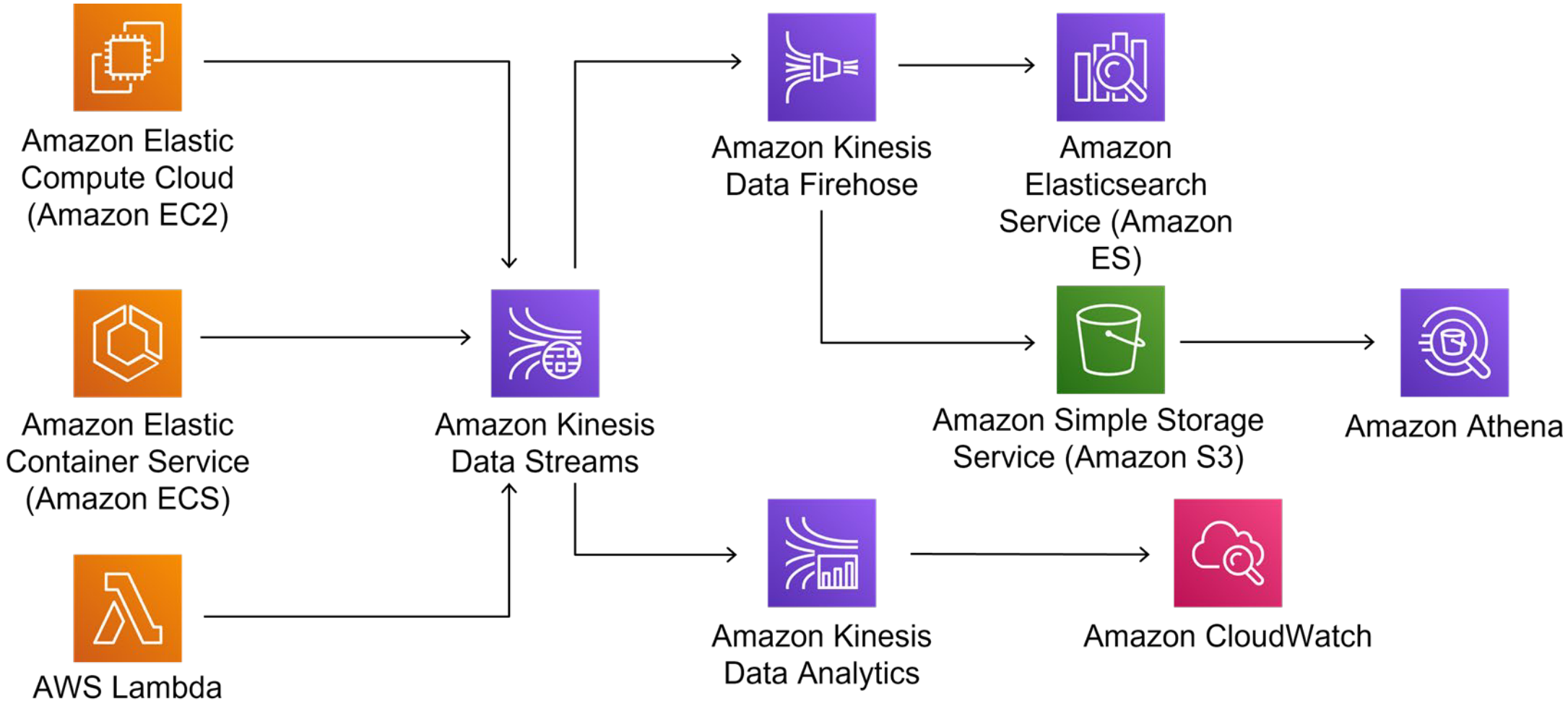
- Deliver streaming data to data stores without writing consumer applications for your stream
- Automatically convert incoming data to open and standards based before the data is delivered

Amazon Kinesis Data Analytics

- Perform real-time analysis on data in the stream using SQL queries before persisting the data
- Use Apache Flink, Java, Scala, or Python for your analysis application

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Kinesis Data Streams example



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Kinesis API operations

`CreateStream:` Creates a stream. Requires ShardCount and StreamName.

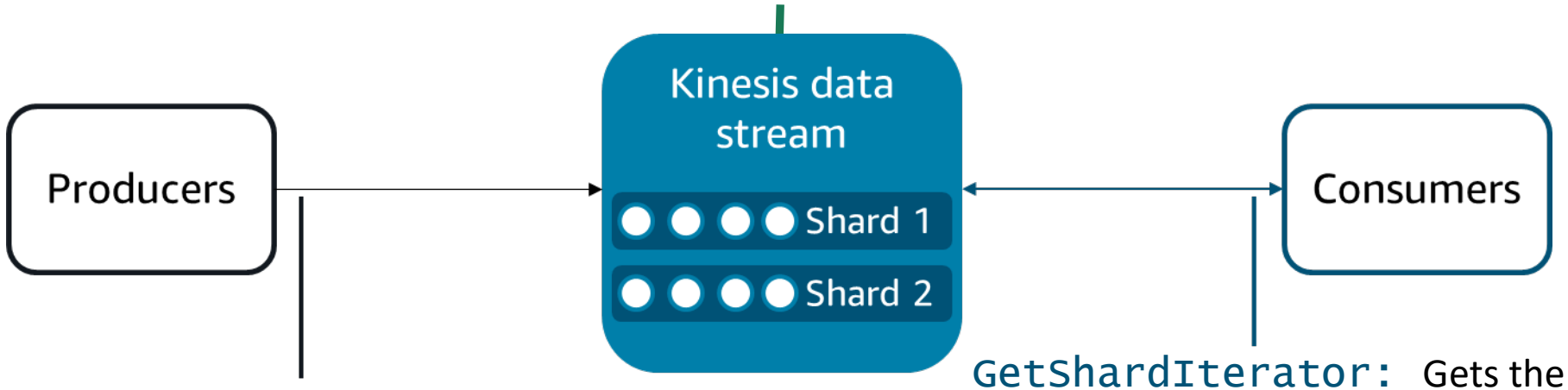
`DeleteStream:` Deletes the named stream and all of its data shards.

`DescribeStreamSummary:` Provides summary information about a named stream.

`UpdateShardcount:` Modifies the number of shards on the stream.

`MergeShards:` Merges two shards to reduce the stream's capacity.

`SplitShards:` Splits a shard into two shards to increase the stream's capacity.



`PutRecord/PutRecords:` Writes a record or records to the stream. Includes a partition key that determines which shard the record(s) will be written to.

`GetShardIterator:` Gets the shard iterator, which tells you the shard position from which to start reading records.

`GetRecords:` Reads records from the stream, starting from a specified shard iterator.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Examples using the AWS CLI for basic tasks

Create a stream named MyStream with one shard

```
aws kinesis create-stream --stream-name MyStream --shard-count 1
```

Check whether your new stream is ready to be used

```
aws kinesis describe-stream-summary --stream-name MyStream
```

Put a test record on the stream

```
aws kinesis put-record --stream-name MyStream --partition-key 123 --data testdata
```

Get the starting position to read the stream, then use the output value to get records

```
aws kinesis get-shard-iterator --shard-id shardId-000000000000 --shard-iterator-type
TRIM_HORIZON --stream-name MyStream

aws kinesis get-records --shard-iterator AAAAAAAAAAHSywl…
```

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Building producers and consumers

Feature	Kinesis Producer Library (KPL)	Kinesis Client Library (KCL)	Kinesis API with AWS SDK
Abstracts stream management tasks so you can focus on application logic	yes	yes	no
Handles **producer tasks** such as writing with automatic retries and aggregating records to optimize throughput	yes	no	no
Handles **consumer subtasks** such as load balancing and checkpointing processed records	no	yes	no
Provides the ability to write **producer or consumer applications** using direct interaction with the Kinesis API	no	no	yes
Requires your application code to handle stream management	no	no	yes

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Comparison of queues and streams

	Queues	Streams
Data value	The value comes from processing individual messages.	The value comes from aggregating messages to get actionable data.
Message rate	The message rate is variable.	The message rate is continuous and high volume.
Message processing	Messages are deleted after a consumer successfully processes them.	Messages are available to multiple consumers to process in parallel, and each consumer maintains a pointer but does not delete records.
Example use cases	Financial transactions, product orders	Clickstream data, application logs

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Section 8 key takeaways

- With **Kinesis Data Streams**, you can **ingest, buffer**, and **process streaming data in real time**.
- **Kinesis Data Firehose** and **Kinesis Data Analytics** were designed to **simplify** common data streaming use cases.
- **Producers put records** on to a stream where the records are **stored in shards**, and **consumers get records** off the stream for processing.
- The Kinesis Producer Library (**KPL**) and Kinesis Client Library (**KCL**) **abstract stream interactions** for developers.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab 10.1: Implementing a Messaging System Using Amazon SNS and Amazon SQS

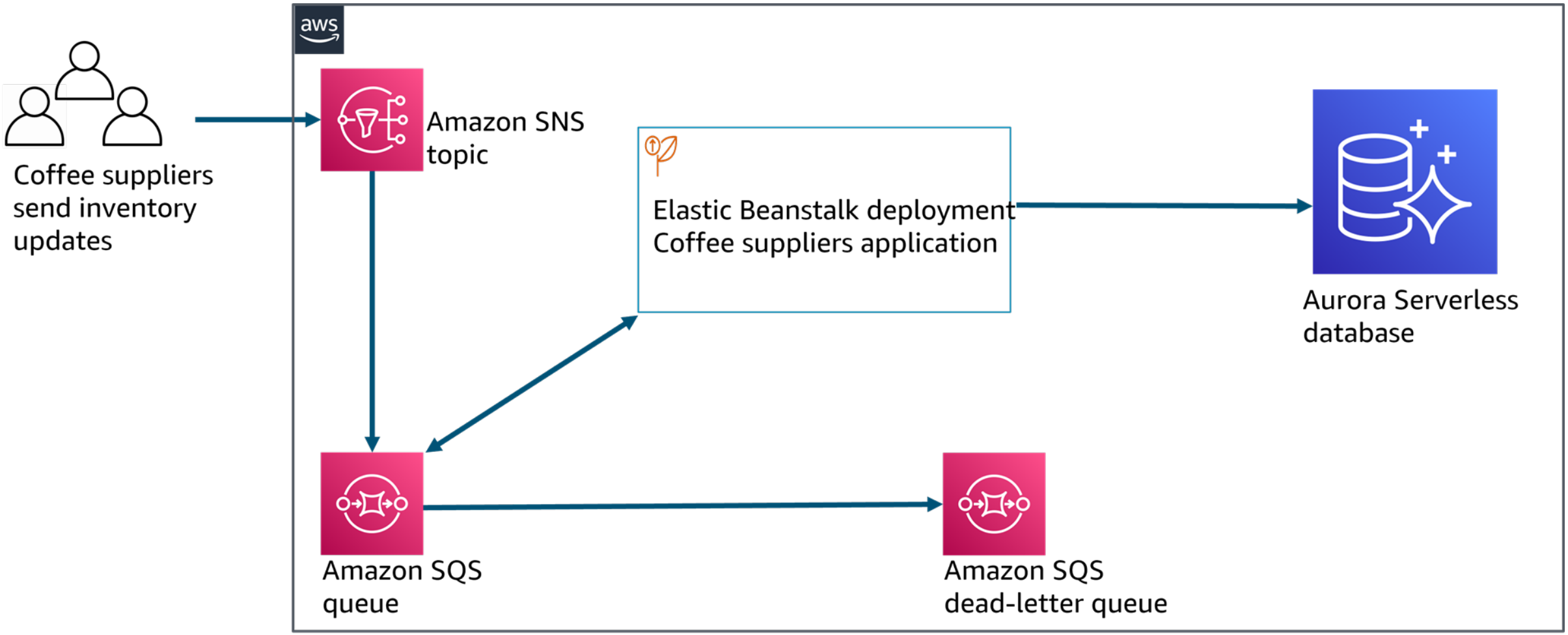
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab: Tasks

1. Preparing the development environment
2. Configuring the Amazon SQS dead-letter queue
3. Configuring the Amazon SQS queue
4. Configuring the Amazon SNS topic
5. Linking Amazon SQS and Amazon SNS
6. Testing message publishing
7. Configuring the application to poll the queue

aws
©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab: Final product



©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

~ 90 minutes

Begin Lab 10.1: Implementing a Messaging System Using Amazon SNS and Amazon SQS

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Lab debrief: Key takeaways

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Module wrap-up

Module 10: Developing with Messaging Services

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Module summary

In summary, in this module, you learned how to do the following:

- Illustrate how messaging services, including queues, pub/sub messaging, and streams, support asynchronous processing
- Describe Amazon SQS
- Send messages to an SQS queue
- Describe Amazon SNS
- Subscribe an SQS queue to an SNS topic
- Describe how Kinesis can be used for real-time analytics

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Complete the knowledge check

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Sample exam question

A developer wants to introduce an asynchronous connection in their application workflow that processes individual banking transactions including deposits and withdrawals. Transactions must be handled in the order they arrive. Which method would meet the developer's needs?

Identify the key words and phrases before continuing.

The following are the key words and phrases:

- **That processes individual banking transactions**
- **The order**

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Sample exam question: Response choices

A developer wants to introduce an asynchronous connection in their application workflow **that processes individual banking transactions** including deposits and withdrawals. Transactions must be handled in **the order** they arrive. Which method would meet the developer's needs?

Choice	Response
A	Send transaction messages to a standard SQS queue.
B	Put transaction messages on to a Kinesis data stream.
C	Send transaction messages to a FIFO SQS queue.
D	Configure Amazon ECS with a cluster of EC2 instances that run Docker containers.

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Sample exam question: Answer

The correct answer is C.

Choice	Response
A	Send transaction messages to a standard SQS queue.
B	Put transaction messages on to a Kinesis data stream.
C	**Send transaction messages to a FIFO SQS queue.**
D	Configure Amazon ECS with a cluster of EC2 instances that run Docker containers.

aws

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Thank you

Corrections, feedback, or other questions?
Contact us at https://support.aws.amazon.com/#/contacts/aws-academy.

©2023, Amazon Web Services, Inc. or its affiliates. All rights reserved.